

A Domain-Specific Computer Language for Market Monitoring: from Domain Logic to SAS Code

Eric C. Grasby, SPP; Daniel Berleant, UA Little Rock, Maria CB Gingras, SPP

ABSTRACT

This paper presents a proof-of-concept design and implementation of a domain-specific language (DSL) for use in energy market monitoring. Energy markets are complex systems that enable market participants to economically trade energy at wholesale rates. Market monitors analyze these markets and screen for manipulative behavior, often using SAS® for data extraction, transformation, and analysis.

While SAS is a well-established programming language for data processing, many market monitors are not expert computer programmers. DSLs can enable non-programmers to solve problems without requiring a background in software development. This paper presents a DSL that aims to help a monitoring analyst declare . It then transpiles their output into valid SAS code, leveraging SAS data sets for data management and SAS procedures, DATA step programming, and SAS macros to handle complex functionality.

Finally, this paper illustrates the flexibility of this proposed design by demonstrating how a user can generate SAS code to fulfill common market monitoring tasks. These tasks were identified by a panel of market monitors as part of a Delphi study.

Keywords: *domain-specific language, analyst abstraction, energy market, market monitoring, transpiler*

INTRODUCTION

Energy is a ubiquitous commodity; its generation, transmission, and use is present in almost every aspect of modern society. Commonly recognized as the “world’s largest machine” [1], the North American Bulk Electric System (BES) is one of the most important systems to the lives of virtually every resident of the United States, Canada, and a portion of Mexico. Organized energy markets, which use the BES to transmit electricity, are necessary constructs that allow power producers and traders to reliably and economically deliver energy to consumers.

After the downfall of the former energy giant *Enron Corporation* the Federal Energy Regulatory Commission (FERC) mandated that all organized energy markets in the United States be independently monitored by an energy market monitoring authority. Market monitors are the group of professionals that work to ensure that these markets remain fair, efficient, and open access.

Finish introduction and framing to introduce the idea of a domain-specific language (and explain what a DSL is)

BACKGROUND

The business problem: why a domain-specific language?

Systems such as SAS already provide an abstraction layer for data processing – why propose to build a DSL on top of SAS at all? The answer to this question lies in the energy data landscape itself. Although the BES is operationally complex, the surrounding data infrastructure is even more fragmented. Market monitors routinely interact with disparate systems, siloed databases, undocumented schemas, and numerous other information quality issues that impede their analysis.

Many of these issues are *semantic problems*, requiring an individual to encode their domain expertise into SAS or SQL scripts. A prevalent example in energy markets involves how timestamps are interpreted across time zones. Because energy market transactions span across time zones, timestamps on these transactions may be recorded in local time, adjacent market time, or Coordinated Universal Time (UTC/GMT). If the time zone metadata is not readily apparent, analytical queries may produce *fundamentally incorrect results*, despite being written as *syntactically valid*.

Modern analytics platforms provide mechanisms for performing these conversions, but they tend to rely on the user understanding and correctly applying the domain-level semantics associated with the data (using the appropriate time zone, for example). By leveraging metadata and domain-aware rules, the DSL can transparently handle these operations without requiring the user to manually account for such details.

The proposed DSL is not intended to replace existing analytics platforms, like SAS, but instead provides a semantic layer that mitigates these information quality risks.

Why SAS®?

Using SAS as the target language for this DSL comes with several advantages. While the design and implementation of this language provide a parser and language transpiler to effectuate the execution of the language, there are other components that are required to provide users with a functional analysis system. These main components include a data processing engine, a reliable data typing scheme, and capabilities for data visualization and basic cartography.

The SAS ecosystem satisfies each of these system requirements. BASE SAS and SAS/ACCESS provide mechanisms for processing data, as well as importing and exporting that data from a variety of external systems (including relational databases). When data is loaded from a source system, SAS houses that data in SAS datasets. These datasets, in turn, provide a reliable and universal API for data processing across all other SAS components. BASE SAS also ships with a data typing system, so the upstream DSL does not have to enforce strict typing as part of the syntax (which translates to less cognitive burden on the user).

Additionally, the Output Delivery System (ODS) that ships with SAS provides first-class citizens for generating and formatting both data visualization graphs and spatial maps. The ODS principally provides this functionality through the SGMAP and SGPLOT procedures, respectively.

With each of these infrastructural components available within one platform, the DSL can remain focused on allowing the user to write intuitive data logic. It then offloads any infrastructural complexity to SAS and return results and error messages back to the user.

REQUIREMENTS SOLICITATION

To identify relevant use cases during the implementation of this DSL, the researchers chose to proctor a Delphi study¹. Eligibility to participate in this study was determined by only two (2) requirements:

1. The participant must be over the age of 18
2. The participant must be a member of the Energy Intermarket Surveillance Group (EISG)²

These eligibility requirements were introduced to both adhere to responsible research ethics³ and to establish a definition of “expert opinion” provided by each participant. Plainly, this ensured that the survey questions posed in this study were answered by the same individuals that may use a purpose-made DSL for energy market analytics. The study was conducted in two (2) rounds, where a round corresponds to a group of survey questions that were provided to the participants. These included a list of free-form text responses, multiple choice questions, and questions provided in a Likert format.

Panel Composition

At the time of administering this study, the EISG held a roster of 131 active members. Round #1 yielded a roughly 15% response rate (19 confirmed individuals participated in the first round). Only 11 of the original 19 participants elected to respond to round #2 (roughly a 58% retention rate of the original participants). Of the panelists, 16 of the EISG members were US-based market monitors. The remaining 3 worked for an internationally based market monitoring unit. Table 1 displays the professional role mix of the panelists, indicating that market monitoring analysts make up the largest portion of the panelists, with a mix of other roles including management, economists, data scientists, and a developer.

Role	Count of Role in Panel	Percentage (%)
Market Monitor (Analyst)	13	68%
Management	4	21%
Economist	3	16%
Data Scientist	2	11%
Developer/IT	1	5%

¹ A Delphi study “is a research survey technique used as a way of collecting data from respondents within their domain of expertise.” Put plainly, it is a panel of experts that anonymously respond to survey questions. A researcher then analyzes their results to arrive at a consensus that serves as the solution to a problem. [2]

² The Energy Intermarket Surveillance Group (EISG) is the international, professional, conference of energy market monitors. Any given professional is a member of the EISG by virtue of working for an MMU that is an organizational member of the EISG.

³ The research protocol for this Delphi study was approved by the UA Little Rock Institutional Review Board. The research protocol and copy of the waiver of informed consent can be found in this GitHub repository: <https://github.com/echarlesgrasby/dissertation/tree/master/ext>.

Table 1. Professional Role Mix of Delphi Panel⁴

Discuss the tenure range of the panelists to give credit to why the participants were positioned as experts to provide their opinions.

Tenure Range	Participant Count	Percentage (%)
0-2 years	3	16%
5-10 years	6	32%
10-15 years	7	37%
15-20 years	1	5%
Greater than 20 years	2	11%

Table 2. Tenure range of Delphi Panel

Derived requirements and use cases

- Data visualization and calculation of metrics
- Basic cartography
- Guided analysis of data (summary stats, data dict, etc.)

Test Data

⁴ Participants were allowed to select more than one role that applied to them (in Table 1), so those that chose (Management) are also represented by other roles that they once performed.

DOMAIN-SPECIFIC LANGUAGE DESIGN

Design goals and constraints

Grammar specification

Discuss Lark grammar and provide an example of what you have so far...

```
1  //-----
2  //FILENAME:      summarize_dataset.lark
3  //AUTHOR:        Eric C. Grasby, MSIQ
4  //INIT DATE:     2026-05-27
5  //PURPOSE:       contains a grammar for allowing a user to generate a basic
6  //              "about this dataset" summary
7  //TRANSPILER TARGET: BASE SAS (PROC SQL; PROC PRINT;)
8  //-----
9
10
11 start: dataset summary
12
13 location: LIBNAME
14 dataset: "DATASET" LIBNAME NAME ";"
15 summary: "SUMMARY" "{" summary_operation+ "}"
16
17 summary_operation: count_op
18                  | mean_op
19                  | min_op
20                  | max_op
21                  | peek_op          //if peek_op is used, call PROC PRINT; in the SAS domain
22
23 count_op: "COUNT" ";"
24 mean_op: "MEAN" NAME ";"
25 min_op: "MIN" NAME ";"
26 max_op: "MAX" NAME ";"
27 contents_op: "CONTENTS" ";"
28 peek_op: "PEEK" ";"
29
30 NAME: /[a-zA-Z_][a-zA-Z0-9_]*/
31 LIBNAME: /[a-zA-Z_][a-zA-Z0-9_]*/
32
33 %import common.WS
34 %ignore WS
35
36 // -----
37 // Example mlang code stored in separate .mlang text file
38 // -----
39 // DATASET WORK TRADES;
40 //
41 // SUMMARY {
42 //     COUNT;
43 //     MEAN price;
44 //     MIN price;
45 //     MAX price;
46 //     PEEK;
47 // }
48 //
49 // END OF FILE
```

Figure 1. Grammar for a "summarize dataset" feature

Example DSL statements

TRANSPILER IMPLEMENTATION

To implement this domain-specific language, the researchers used two (2) main technologies: Python (including the Lark parser framework⁵) and the SAS® OnDemand for Academics (SODA) environment. The authors created a

⁵ The terms *lexing* (tokenization) and *parsing* refer to distinct operations in computer science for working with text. A lexer takes an input string and breaks it up based upon a delimiter (usually a whitespace character) into atomic *lexemes* (e.g. "Brenda was the best programmer amongst us" would be "lexed" as : ["Brenda", "was", "the", "best", "programmer", "amongst", "us"]). *Parsing* then takes this tokenized text and applies syntactic rules to it to develop a *parse tree* (i.e. identifying "Brenda" as the subject of the sentence and "was"

Python library that defines the grammar of the DSL, builds the lexer, parser, and transpiler, and then outputs valid SAS code. The code is then uploaded to SODA to check for execution errors and to retrieve any results.

Parser and transpiler architecture

Transpiler process flow shown below:

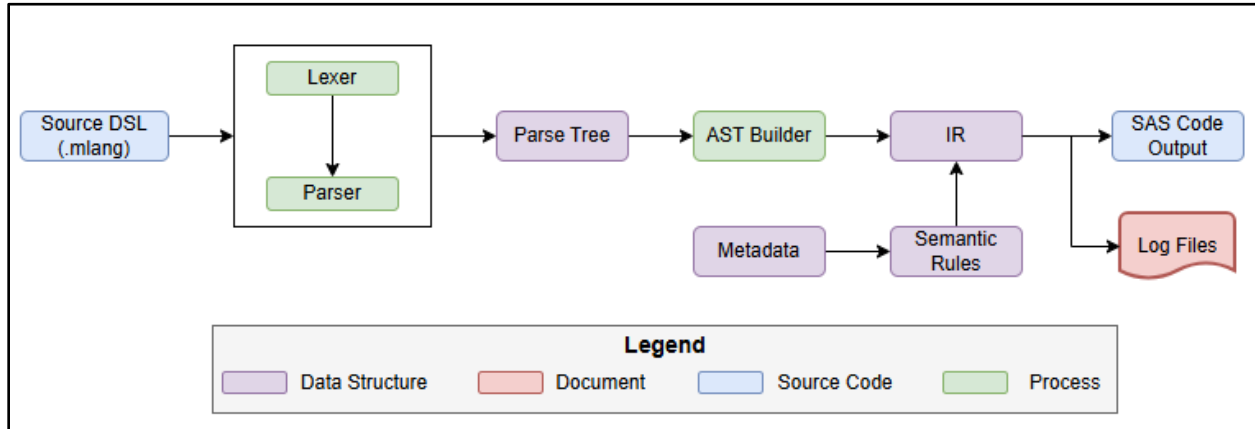


Figure 2. Source DSL to SAS code transpiler process

Transpiler strategy

The screenshot shows a SAS environment with a code editor, a log window, and a results window. The code editor contains the following SQL code:

```

1 PROC SQL;
2 CREATE TABLE WORK.DATASET_FROM_DSL AS
3 SELECT COUNT(1) AS ROW_COUNT,
4 AVG(msrp) AS AVG_msrp,
5 MIN(msrp) AS MIN_msrp,
6 MAX(msrp) AS MAX_msrp
7 FROM SASHELP.CARS;
8 QUIT;
9
10 PROC PRINT DATA=SASHELP.CARS (OBS=10); RUN;
11

```

The log window shows the output of the transpiler process:

```

$ python test_parser_and_transpiler.py
Initial parse tree is:
start
dataset
SASHELP
CARS
summary
summary_operation
count_op
summary_operation
mean_op msrp
summary_operation
min_op msrp
summary_operation
max_op msrp
summary_operation
peek_op

Output SAS code is:
PROC SQL;
CREATE TABLE WORK.DATASET_FROM_DSL AS
SELECT COUNT(1) AS ROW_COUNT,
AVG(msrp) AS AVG_msrp,
MIN(msrp) AS MIN_msrp,
MAX(msrp) AS MAX_msrp
FROM SASHELP.CARS;
QUIT;
PROC PRINT DATA=SASHELP.CARS; RUN;

```

The results window shows the output of the SQL query:

ROW_COUNT	AVG_msrp	MIN_msrp	MAX_msrp
428	32774.85514	10280	192465

Three steps are highlighted in the image:

- Step 1: User invokes transpiler (indicated by a red box pointing to the command prompt window).
- Step 2: Move output code into SAS environment (indicated by a red box pointing to the PROC PRINT statement in the code editor).
- Step 3: Retrieve results (indicated by a red box pointing to the results window).

Figure 3 - Sample execution of transpiler process

as the verb).

Handling complex functionality

EVALUATION OF RESULTS

Use case demonstrations

Verification of correctness

CONCLUSION

Conclusions...

Limitations

Given that this paper provides proof-of-concept for this DSL, there are several limitations with the current implementation of this solution, namely:

1. The execution model of this DSL exists solely as preprocessor directives
2. Use of the resultant SAS code requires manual intervention

Future Work

This paper presents a practical foundation for translating a custom domain-specific language into valid SAS code. Given the identified limitations of this current approach, the authors believe that the following areas are possible opportunities for future work on this system. These future work topics include: integrating language server protocol support, enhancing the current error handling feedback from the transpiler layer, and integrating the generated SAS output with a SAS runtime environment.

Language Server Protocol (LSP) support

Enhanced error handling

Integration with a SAS runtime environment

REFERENCES

References are in-progress

[1] Oak Ridge National Laboratory, "Nils Stenvig: Exploring the world's largest machine [ORNL]." <https://www.ornl.gov/blog/nils-stenvig-exploring-worlds-largest-machine>.

[2] Ruslan Mitkov, *The Oxford handbook of computational linguistics*. Oxford: Oxford University Press, 2004.

[3] S. Chuenjitwongsa, "How to: Conduct a Delphi Study," *Cardiff University*, 2017. https://www.cardiff.ac.uk/_data/assets/pdf_file/0010/1164961/how_to_conduct_a_delphistudy.pdf

ACKNOWLEDGMENTS

We would like to acknowledge SPP, the Energy Information Authority (EIA), and DegreeDays.net for providing sources of publicly available data which made the construction and evaluation of this project possible. Readers are encouraged to visit the website, for each respective organization, for more information on sources of energy data:

- SPP (née Southwest Power Pool) -- <https://portal.spp.org/groups/integrated-marketplace>
- EIA -- <https://www.eia.gov/electricity/>
- "BizEE" Degree Days -- <https://www.degreedays.net/>

Additionally, Eric would like to acknowledge both Mark Rouse and Bartosz "Bart" Jabłoński for inspiring excitement within him for learning SAS.

RECOMMENDED READING

Recommended reading in progress...

- Steven Stoft's *Power System Economics: Designing Markets for Electricity*
- I. Khorlo, "Lint: A SAS Program Checker," in *Pharmaceutical Users Software Exchange 2017*, Lex Jansen, Oct. 2017. Available: <https://www.lexjansen.com/phuse/2017/ad/AD08.pdf>

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author(s) at:

Eric C. Grasby

Senior Market Monitor, SPP, Inc.

Doctoral Candidate, UA Little Rock, Department of Information Science

ecgrasby@ualr.edu

<https://linkedin.com/in/echarlesgrasby>

Dr. J. Daniel Berleant

Professor, UA Little Rock, Department of Information Science

jdberleant@ualr.edu

<https://www.linkedin.com/in/daniel-berleant-4938451/>

Dr. Maria CB Gingras

<https://www.linkedin.com/in/mariacbrun/>

SAS and all other SAS Institute Inc. product or service names are registered trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Other brand and product names are trademarks of their respective companies.

APPENDIX

See <https://github.com/echarlesgrasby/dissertation-grammar> for version-controlled copies of any of code depicted below.

Monitor Language ("Mlang") Grammar – Version 1

Template for mlang project configuration file

Template for mlang metadata file

Template for mlang link file